From Fold to Function: Dynamic Modeling and Simulation-Driven Design of Origami Mechanisms

Anonymous for review

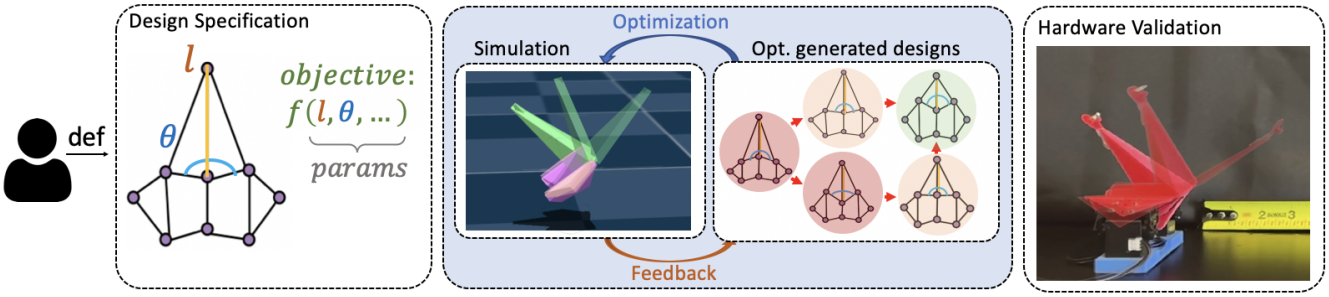

Fig. 1. An overview of the proposed framework: users define their origami designs through a graphical user interface (GUI), which automatically converts the specifications into a MuJoCo compatible simulation model. The simulation environment then generates data through dynamically interacting with the modeled mechanism which can be used as feedback for design optimization. We validate the final optimized configuration by fabricating the design in hardware and demonstrating improved performance compared to non-optimized variants.

***Abstract*— Origami-inspired mechanisms can transform flat sheets into functional three-dimensional dynamic structures that are lightweight, compact, and capable of complex motion. These properties make origami increasingly valuable in robotic and deployable systems. However, accurately simulating their folding behavior and interactions with the environment remains challenging. To address this, we present a design framework for origami mechanism simulation that utilizes MuJoCo's deformable-body capabilities. In our approach, origami sheets are represented as graphs of interconnected deformable elements with user-specified constraints such as creases and actuation, defined through an intuitive graphical user interface (GUI). This framework allows users to generate physically consistent simulations that capture both the geometric structure of origami mechanisms and their interactions with external objects and surfaces. We demonstrate our method's utility through a case study on an origami catapult, where design parameters are optimized in simulation using the Covariance Matrix Adaptation Evolution Strategy (CMA-ES) and validated experimentally on physical prototypes. The optimized structure achieves improved throwing performance, illustrating how our system enables rapid, simulation-driven origami design, optimization, and analysis.**

I. Introduction

Origami-inspired design has evolved from an ancient Japanese artistic practice to a powerful engineering design paradigm [1]. By designing complex motion into a flat sheet of material, origami principles offer exceptional scalability and manufacturing efficiency, allowing devices to be shipped in a flat or folded form and deployed on demand [2]. Building on these advantages, origami mechanisms enable compact, lightweight, and reconfigurable systems in diverse domains, such as biomedical devices [3]–[5], space robotics [6], [7] and educational tools [8], [9]. In robotics, origami-inspired mechanisms have been integrated into a variety of designs capable of crawling [10], [11], walking [12], [13], jumping [14], and grasping [15]–[17].

Despite these advances, the design of functional origami mechanisms remains largely intuitive and reliant on physical iteration. Modeling the relationship between crease patterns, material properties, and 3D kinematics requires solving for behavior over high-dimensional, nonlinear parameter spaces [18], making analytical prediction and manual design exploration computationally prohibitive or mathematically complex. Achieving a desired motion or performance often requires repeated cycles of geometric tuning, fabrication, and testing, as small variations in crease geometry, fold angle, or sheet stiffness can lead to different mechanical behaviors [19]. Recent studies have applied optimization techniques to improve the performance of origami-inspired robotic systems. For example, Chen et al. [20] employed a genetic algorithm to optimize a foldable pneumatic actuator over multiple geometric parameters, achieving enhanced actuation performance. However, their approach relied on a detailed system-specific mathematical model of the volume, pressure, and torque of their actuator. Cao et al. [21] proposed an inverse-design framework for an origami gripper that adjusts crease-pattern parameters to optimize for better grasping metrics, which proved very effective for task-specific performance tuning. Yet, the framework relies on analytical modeling of a particular Yoshimura-based geometry, making it less generalizable to other origami morphologies. Zhu et al. [22] proposed a bio-inspired trajectory optimization framework for a multi-locomotion origami robot, combining dynamic and kinematic modeling with graph-based path planning to optimize gaits for crawling and swimming. Although this work demonstrates strong system-level integration and effectively addresses its target locomotion tasks, its optimization is closely coupled to the robot's specific morphology and motion modes. More recently, optimization methods have been extended to reconfigurable

origami-inspired manipulators, where physical reconfiguration parameters and control policies are co-optimized through reinforcement learning to enable adaptive behavior across tasks [23].

While these works showcase the potential of optimization in origami robotics, they depend on specialized forward models or analytical approximations that limit generalization to other morphologies or materials. Most existing methods require derivation of a custom kinematic model for each mechanism, a process that is both time-consuming and not generalizable. While these models are capable of capturing local deformation or actuation behavior of a specific pattern with high physical fidelity, they typically cannot simulate how the structure interacts with external objects or dynamic environments without significant modification. As a result, extending these frameworks to new geometries, materials, and interactions, or adding new tasks, might require rebuilding the model or re-deriving its governing equations from scratch. This motivates the development of a more generalizable and accessible framework, one that allows designers to test and refine origami mechanisms directly within a realistic physics simulation environment, enabling rapid iteration and performance-driven design exploration.

To address this gap, we present a physics-based origami simulation framework that integrates geometric design, dynamic modeling, and performance optimization within a single framework. Our system represents origami mechanisms as graphs of interconnected deformable panels, simulated using MuJoCo's deformable-body physics engine. The contributions of this work are:

- A physics-based simulation framework for origami mechanisms that integrates geometric design, dynamic modeling, and optimization, enabling contact-rich and actuation-driven simulations through MuJoCo's deformable-body system.
- A graph-based representation and graphical user interface (GUI) that allow users to intuitively define crease patterns, assign material and actuation properties, and automatically generate physically consistent MuJoCo models from 2D schematic inputs.
- A case study demonstrating simulation-driven optimization and hardware validation, where the framework is used to optimize an origami catapult mechanism via CMA-ES, achieving improved throwing performance and consistent ranking between simulation and physical experiments.

An overview of the complete framework, spanning user design input, automatic model generation, simulation, and hardware validation, is illustrated in Fig. 1.

II. Related Work

Recent advances in computationally efficient origami simulations focus on geometric and kinematic modeling of fold patterns, emphasizing mathematical tractability and visualization of crease patterns. For example, Demaine et al. established rigorous geometric models for understanding foldability and motion in origami-based systems [24], while interactive tools like Origamizer [25], [26], Rigid Folding Simulator [27] and Freeform Origami [28] enabled users to visualize folding trajectories that satisfy loop-closure constraints and fit any triangulated 3D surface using origami creases. These tools allow for rapid design exploration and visualization of the origami mechanism, yet they are limited to quasi-static or kinematic folding and do not capture dynamic behaviors or actuation effects.

Methods have also been developed to incorporate mechanics and multiphysics-based modeling to simulate deformation, actuation, and nonlinear material responses. Ghassaei et al. [29] introduced a GPU-accelerated origami simulator that allows real-time interaction on a web-browser and visualization of material strain, while Sung et al. [30] developed a computational design tool that enables automated creation of compliant mechanisms from origami patterns. Multiphysics simulators have also been developed to couple thermal, magnetic, or fluidic actuation to folding behavior [31]. These advances bridge the gap between geometric models and real-world physical interactions. Building on this progress, our framework aims to introduce a physics-based origami simulation environment using MuJoCo, a high fidelity physics simulator, which enables dynamic, contact-rich modeling of origami mechanisms interacting with rigid bodies, actuators, and sensors within a unified system [32].

III. Methodology

In this section, we first present a graphical user interface (GUI) [33] developed to take intuitive definitions of an origami crease pattern and render the corresponding origami mechanisms in the physics simulator MuJoCo [32]. We then showcase example mechanisms created using this framework, from their fold definitions to how they behave in the simulation. Finally, we use select mechanisms to show 1) the simulation platform as a design tool that allows quick design iteration by sweeping through a range of design parameters, and 2) the optimization in simulation of an origami design using an evolution algorithm, Covariance Matrix Adaptation Evolution Strategy (CMA-ES), and a validation of the resulting design in hardware.

A. Graphical User Interface for Design Rendering

To facilitate the creation of origami mechanism designs, we developed a custom GUI that allows users to define the crease pattern of an origami mechanism abstracted as a two-dimensional graph structure. First, users define the vertices of the graph — key points used to specify boundaries and creases. Next, users add lines that connect the key points together and specify whether each line is a crease line, which means that the line will act as a hinge between the two adjacent panels, or, a boundary line, which defines the external boundary of a panel. To define the movement of panels around a crease, users assign properties to key points on the panels on each side of the crease line. The assignment defines the translational degrees of freedom on the x-, y-, or z-axis that each key point can have. Users can also designate key points as actuation points, specifying actuation along a

chosen global axis (x, y, or z) in the simulation environment. Once the key points and lines form an enclosed geometry, the user defines panels by clicking within the closed shapes. All of the above user definitions are automatically converted into MuJoCo's XML-based MJCF format, enabling simulation of the corresponding deformable structure.

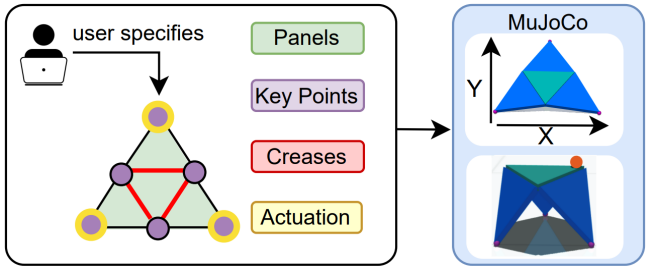

Fig. 2. (Left) purple circles show the key points, yellow highlights key points with defined actuation, black lines are boundary edges, and red lines are creases. Our algorithm converts input from the GUI (left) into MuJoCo XML format to simulate the specified components (upper right), and by using the actuated key points at the very tip of each arm, we can raise and move the mechanism (lower right), dynamically interacting with a rigid sphere placed on the top panel of the mechanism.

An example input-output pair is shown in Fig. 2. From the user definition provided to the GUI, we generate an origami blueprint of connected 2D panels. This allows the 2D sheet to exhibit the desired degrees of freedom determined by the specified folds and actuators. At this point, the origami mechanism is fully defined in the GUI and can be automatically exported as an XML file compatible with MuJoCo for rendering and simulation. When the user loads the exported XML file into MuJoCo, the mechanism appears as a flat sheet resting on the simulated ground plane under gravity. The environment includes a ground plane that constrains the motion of the structure through collision and friction. Because the actuation axes have already been defined in the GUI, users can now apply forces or control input directly to the actuated key points to fold the structure into its intended 3D configuration. The user can also reposition key points by modifying their coordinates in the XML file. In the example shown, this can be done by elevating the key points that form the central triangle. Alternatively, users can define the key points at the tips of the outer triangles to be actuation points, enabling the mechanism to fold and emerge into a 3D configuration from the 2D blueprint. This completes the basic transition from a 2D crease pattern to the 3D mechanism. Since we render the mechanism in a physics environment, additional objects can be introduced into the environment; in this example, a sphere is placed on the top panel of the mechanism, dynamically responding to the movement of the structure rolling across its surface.

To clarify panel definition in detail, we choose the smallest enclosing cycle that includes at least one crease, then we sort the key points within each panel counterclockwise around their centroid to ensure consistent orientation prior to triangulation. Unlike convex hull algorithms, such as Graham Scan [34], which discard interior points, our method retains all key points to preserve the structure of the original panel. This ensures that the panels are well-formed, simple polygons suitable for further meshing. Preserving all key points within each panel is crucial for accurately capturing local geometric variations, which allows the resulting mesh to have some deformability in simulation. In addition, retaining this vertex-level structure lays the groundwork for modeling panels with varying material properties in future extensions of the framework. Each user selection results in the definition of one panel, and the process is repeated until all desired panels are specified.

Algorithm 1 Panel Detection and Triangulation

Require: Graph $\mathcal{G}$ with vertices, edges, and fold labels
1: **while** user clicks within a region **do**
2: Detect minimal cycle enclosing the click position that contains at least one fold edge
3: Sort the vertices of the detected cycle counterclockwise
4: Record the panel defined by the sorted cycle
5: **end while**
6: **for** each recorded panel **do**
7: Offset collinear vertices to avoid degenerate triangles
8: Apply constrained Delaunay triangulation, preserving fold edges as fixed constraints
9: **end for**
Ensure: Triangulated mesh structure for MuJoCo flexible-body simulation

Once the panels are constructed, we apply Delaunay triangulation [35] to each panel to generate a triangular mesh. Delaunay triangulation maximizes the minimum angle across the triangles, avoiding skinny triangles that can cause numerical instability during simulation. However, to triangulate a polygon, especially using Constrained Delaunay Triangulation, the polygon needs to be strictly simple, which requires that there are no overlapping edges and no three consecutive points perfectly collinear within one panel. The first requirement is automatically guaranteed as we do not store replicated edges. To ensure the second requirement, before triangulation, we apply a minor perturbation to collinear key points to prevent degenerate triangles during meshing. The triangulation results define each panel by referencing the key points' indices that make up the panel, rather than the modified coordinates of those key points, thus preserving the original mechanism while ensuring numerical stability during the triangulation process. The overall panel construction and meshing process is summarized in Algorithm 1.

After generating the mesh for each face, the origami mechanism is instantiated in MuJoCo as a flexible deformable body, known as a "flex". MuJoCo versions 3.0 and above introduce a physics-based framework for modeling deformable bodies through a flex system. A flex object is represented as a collection of point masses (MuJoCo bodies) connected by elastic deformable elements (edges (1D), triangles (2D), or tetrahedra (3D)) that define its

geometric and mechanical structure. Modeling origami as 2D flex sheets enables the MuJoCo platform to discretize surfaces into triangular elements, that are treated as constant-stress finite element governed by the Saint Venant–Kirchhoff hyperelastic constitutive law [36]. In our framework, each panel of the origami mechanism is modeled as an individual flex object, with the user-defined key points serving as the point-mass bodies, and the triangulation is completed by the algorithm described above. Because flexes act directly on the underlying bodies in the model, panels that share key points implicitly couple through those shared bodies. The motion of these shared points reflects the combined elastic and contact forces from all connected panels, enabling the simulation of dynamic interactions across the origami mechanism.

B. Example Mechanisms & Key features

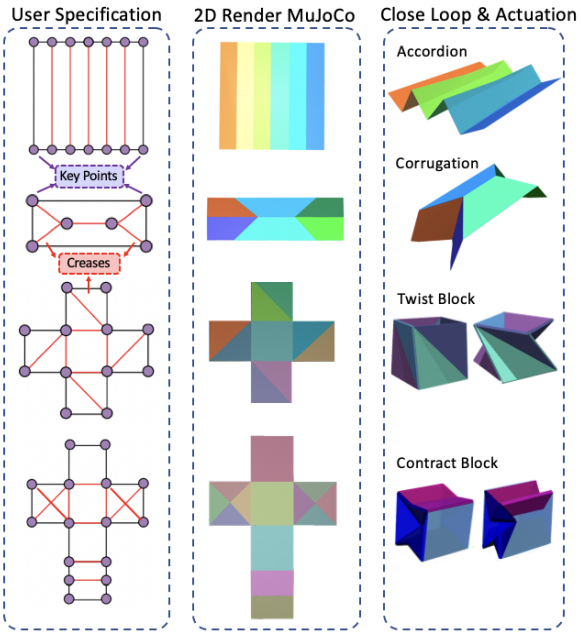

Fig. 3. Examples of standard origami fold patterns processed through our framework. The left column shows user-defined specifications in the graphical interface, where key points (purple) and crease lines (red) define the geometric relationships between panels (actuations omitted for clarity). The middle column shows the corresponding 2D renderings of the generated MuJoCo flex sheets, and the right column shows the resulting 3D closed and actuated mechanisms. From top to bottom: (1) parallel strip folds forming an accordion-like structure; (2) a corrugation pattern with V-shaped valley folds supporting a central mountain ridge; (3) a modular origami actuator block achieving rotational motion through coordinated folding; and (4) a modular origami actuator block achieving horizontal contraction.

To evaluate the modeling capabilities of our framework, we begin by simulating standard origami fold patterns and mechanisms from folding schematics, through our GUI, to simulated mechanisms rendered in simulation. We refer to mechanisms that contain a three-dimensional loop as closed-loop mechanisms; examples include structures such as a box, where four panels form a continuous loop. In Fig. 3, we present this process for both open and closed-looped mechanisms. While both open and closed-looped mechanisms can be defined using our GUI and rendered in simulation, we note that the two closed-loop mechanisms (bottom two in Fig. 3) would require the user to manually merge desired key points in the XML document to ensure the mechanism closes as intended. These examples illustrate the complete workflow of our framework: from defining the crease pattern to generating the resulting three-dimensional mechanism.

Our simulation framework also captures contact-rich interactions between origami mechanisms and their surrounding environment, such as rigid objects or the ground plane. As shown in Fig. 4, the framework can model various behaviors including grasping, throwing, locomotion, and balancing. These examples highlight how MuJoCo's physics engine enables dynamic interaction between origami mechanisms and external objects.

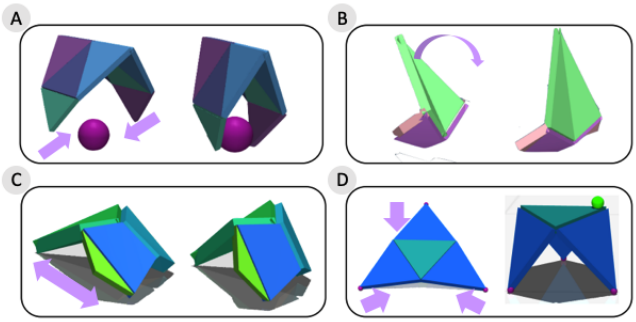

Fig. 4. Examples of origami mechanisms interacting with their environment. (A) An origami gripper that can push, grasp, and release an object. (B) An origami catapult where the top panel rotates upward under lateral actuation. (C) A walker that propels itself forward on the ground plane through cyclic contraction and release. (D) A triangular, legged origami mechanism capable of balancing a rolling sphere by modulating actuation across its legs. The purple arrows indicate the direction of motion of each structure.

C. Using the Framework for Optimization

Beyond serving as a simulation and visualization tool, our framework enables design optimization by allowing structural parameters to be systematically altered and evaluated in physics-based simulation. To demonstrate this capability, we simulate an origami mechanism, catapult, where its dynamic behaviors arise from force propagation and mechanical energy transfer (mechanism B of Fig. 4). In the origami catapult mechanism, lateral actuation is converted into vertical launch force.

To demonstrate rapid optimization over potential designs, we apply the Covariance Matrix Adaptation Evolution Strategy (CMA-ES), a stochastic derivative-free optimization algorithm well-suited for non-linear, non-convex search spaces. CMA-ES maintains a population of design candidates, samples new candidates based on a multivariate normal distribution, and iteratively updates the distribution parameters toward regions of higher performance. An illustration of this design optimization process is shown in Fig. 5. We conduct our optimization in Python using the CMA-ES library with the step size set to $\sigma = 0.025$.

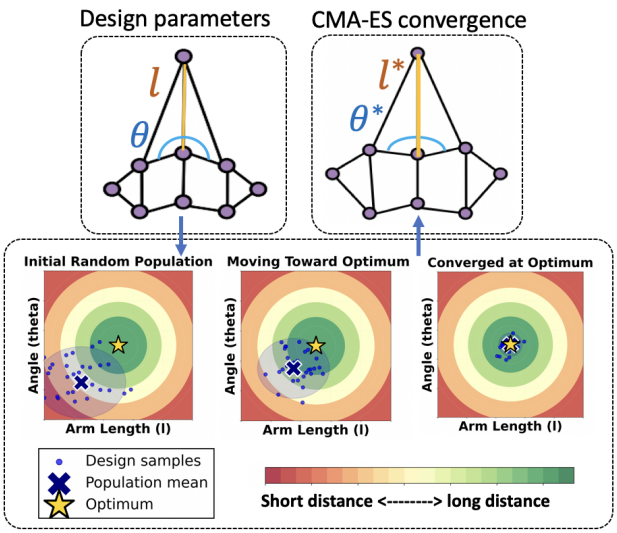

Fig. 5. Optimization of the origami catapult mechanism over two design parameters: (1) the sector angle of the mountain folds (θ) and (2) the length of the throwing arm (l). The top row shows the initial and optimized configurations of the mechanism. The bottom row illustrates the CMA-ES optimization process, where the population of candidate designs progressively converges toward the region corresponding to maximal throwing distance.

IV. Simulation Experiments and Results

We present the design and optimization results of our framework on the catapult mechanism. Two key geometric parameters were varied to generate new designs:

- Fold angle θ of the mountain fold, which is controlled by 2 key points (vertices 2 and 6 in the XML file definition). We vary this parameter by systematically changing the position of vertices 2 and 6, allowing us to search over the range of $\theta \in [100°, 226°]$
- Arm length l, controlled by key point (vertex 10 in the XML file definition). We search over the range $l \in [0.08, 0.18]$ meters.

We first design, define, and render the origami catapult structure using the graphical user interface (GUI) and workflow described in the previous section. We then apply the Covariance Matrix Adaptation Evolution Strategy (CMA-ES) optimization to the simulated catapult mechanism introduced in Section III. The catapult consists of six panels and three primary fold lines at its center, which together enable the throwing motion. The objective of the experiment is to maximize the horizontal throwing distance of a 1 g spherical payload with a radius of 0.01 m. To ensure consistent initial conditions, the sphere is positioned 0.002 m above the tip of the catapult arm which is close enough to the surface to settle onto it under gravity but without initial contact. In the absence of actuation, the sphere simply falls and rolls along the throwing arm. During simulation, a fixed actuation policy applies a single positional command at the 10th simulation timestep (corresponding to the moment when the sphere has settled on the arm), producing a max angular velocity of approximately 2.08 rad/s at the two outermost actuated key points. The projectile's travel distance, measured in both simulation and subsequent hardware experiments, is defined as the horizontal distance between the sphere's initial position and its final resting position along the throwing axis.

To better analyze the optimization results, we also perform a parameter sweep. For each design, we vary the length of the throwing arm of the catapult l as well as the angle of the fold θ. We search over the same ranges: $\theta \in [100, 226]$ degrees and arm length $l \in [0.08, 0.18]$ meters. We iterate over 2880 combinations of parameters θ and l. In both the brute force parameter sweep and the CMA-ES optimization, other than the key points involved in altering these parameters (two vertices controlling the angle and one vertex controlling the arm length), all key points are kept in the same position with the same properties to ensure consistency.

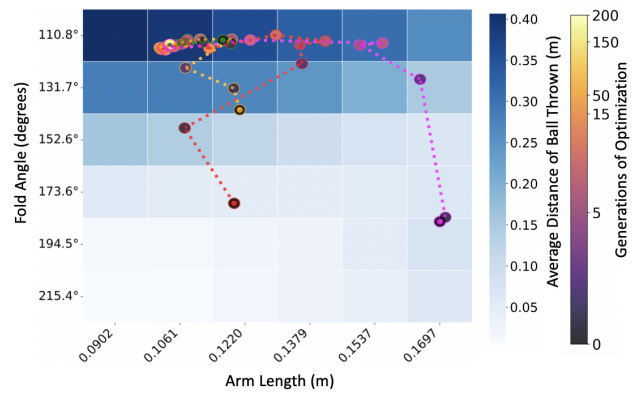

Fig. 6. Heatmap of simulated throwing distance across the design space defined by fold angle (θ) and arm length (l), overlaid with CMA-ES optimization trajectories. The color of each parameter bin in the heatmap represents the average throwing distance of simulated designs within that bin. Colored paths trace the progress of four independent CMA-ES runs over 200 generations, with color indicating the generation number. All trajectories converge toward the same high-performance region and ultimately to the same optimal design, indicated by the white circle at generation 200.

The parameter sweep highlights a clear performance group corresponding to sharper fold angles ($110°$ to $130°$) and moderate arm lengths (0.10 m to 0.13 m), suggesting that insufficiently sharp fold curvature or overly long arms reduce the energy transfer efficiency from lateral actuation to vertical launch. However, because each bin represents the average performance of all designs that fall within its discrete interval, the true optimum may not be captured precisely. Achieving a finer resolution would require sampling a substantially larger number of parameter combinations, rapidly increasing the computational cost of an exhaustive search.

To evaluate sample-efficient design optimization, we ran CMA-ES from randomized initial conditions over the same parameter space. We showcase four such optimization trajectories in Fig. 6 as a colored path overlaid on the heatmap of sweep results. All trajectories converge toward the same region identified by the brute-force search, validating the consistency and robustness of the simulation-based optimization pipeline.

The best-performing configuration identified by CMA-ES corresponds to a fold angle $\theta^* \approx 115.5°$ and arm length $l^* \approx$

0.102 m, achieving a simulated average throwing distance of 0.4717 m.

V. Hardware Validation Experiments and Results

A. Motivation

The motivation of the hardware experiments is to validate the predictive accuracy of our simulation framework by comparing the performance of the optimized mechanism identified in simulation using the CMA-ES optimization against two sub-optimal designs predicted to perform worse in simulation. While we expect quantitative differences due to sim-to-real discrepancies arising from unmodeled material properties and imperfect representation of crease compliance, the relative performance ordering should remain consistent, as the geometry of each design dictates its throwing behavior, which our simulation should capture faithfully.

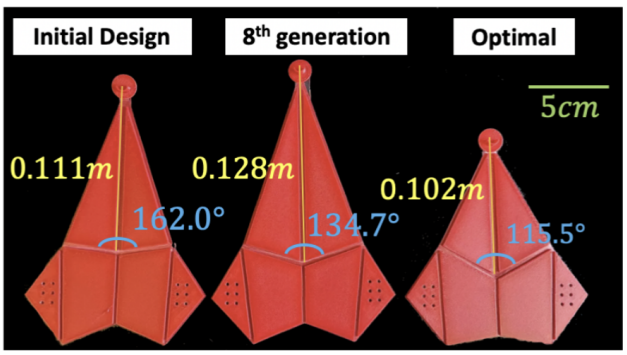

Fig. 7. Three catapult configurations selected for hardware validation. The optimized design corresponds to the best-performing parameters found in simulation. We also included the initial design that the optimization process started with, and the best performing design we found at the 8th generation of the optimization trajectory.

We sampled a single CMA-ES optimization trajectory and recorded three representative designs: the randomly initialized starting design, the best-performing design from the 8th generation, and the final optimal design (Fig. 7). The optimization results demonstrate progressive improvement from the initial to the 8th-generation to the final optimized design, and we expect this performance ordering to be consistent in the real-world validation experiments.

B. Experimental Design & Setup

The three catapult prototypes, corresponding to the initial, 8th-generation, and final optimized design (Fig. 7), were 3D-printed using the same fold geometry and spacing as in simulation. The material we used to print these designs is PETG. The faces are made with thickness of 2 mm to allow for some rigidity that makes the mechanism more robust and less likely to be damaged by actuation, and the folds are made to be 0.2 mm to allow flexible folding behavior.

Each mechanism was mounted, as shown in Fig. 8 to a rigid PLA base and actuated using two Dynamixel motors controlled in position mode. The motors were tightly fitted to a base mount of width 90 mm and rotated synchronously by $75°$ outward to generate the catapulting motion. The motor achieved an average angular velocity of 2.09 rad/s during actuation, which the simulation actuation parameters were tuned to approximate. The projectile used in the hardware experiments weighed 1 g.

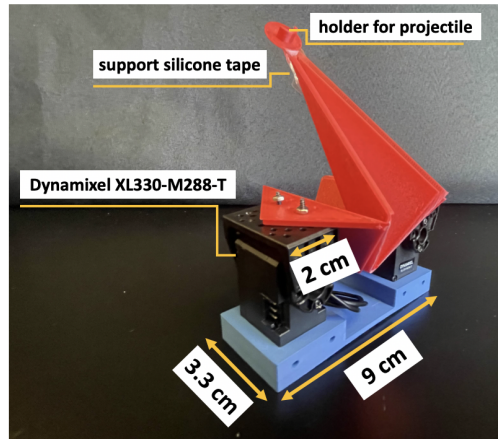

Fig. 8. Hardware setup. Two Dynamixel motors rigidly mounted to a 90 mm base actuate the origami catapult synchronously. Each motor holds a 2 cm mounting basket to which the origami mechanism is fastened using 2 mm M2 screws. Positional control commands rotate both motors $75°$ outward, replicating the actuation profile used in simulation.

C. Hardware Experiment Results

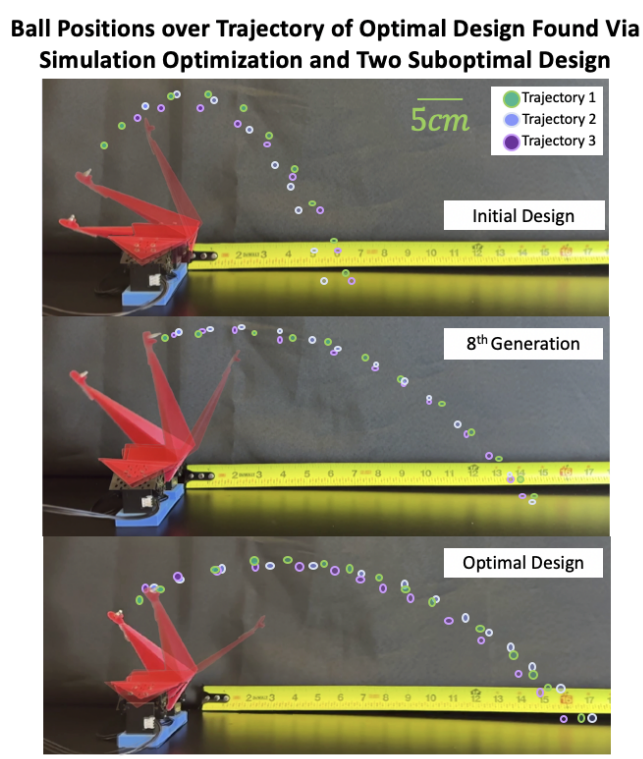

Fig. 9. Projectile trajectories for three catapult designs recorded using frame tracking software, where we mark the path of the projectile using key frames. The optimized configuration achieves the longest throws, consistent with simulation predictions. The tape measure in the background has markings in inches.

Each design was tested for three trials under identical actuation parameters. The mean and standard deviation of

projectile travel distance are summarized in Table I. As predicted by simulation, the optimized design achieved the greatest throwing distance, followed by the two sub-optimal variants.

TABLE I
MEAN DISTANCE PROJECTILE TRAVELED IN HARDWARE VALIDATION EXPERIMENT

Results	**Initial Design**	**8th Gen**	**Opt Design**
Mean Dist (m)	0.308	0.529	0.570
Std Dev (m)	4.29×10^{-2}	5.29×10^{-3}	1.81×10^{-2}

The projectile trajectories recorded by a camera using slow motion at 240 frames per second are shown in Fig. 9, illustrating consistent ordering of performance between designs. The waypoints were marked using the trajectory of the projectile captured in the videos.

To further assess simulation fidelity, Table II compares the mean throwing distances obtained in simulation and hardware, along with percentage differences. Although absolute values differ due to unmodeled factors such as air resistance and material imperfections, the relative ranking of the designs remains consistent across both domains, demonstrating the validity of our simulation framework for comparative design evaluation.

TABLE II
COMPARISON OF THROWING DISTANCE BETWEEN SIMULATION AND HARDWARE WITH SIM TO REAL DIFFERENCE AND PERFORMANCE IMPROVEMENT

Design	**Sim (m)**	**Real (m)**	**Diff (%)**
Initial Design	0.1084	0.308	64.8%
8th Gen	0.4277	0.529	19.1%
Optimal Design	0.4717	0.570	17.2%

The experimental results corroborate the simulation predictions: the design identified as optimal in simulation maintained superior performance in physical testing, while the sub-optimal configurations produced consistently shorter launch distances. Although quantitative discrepancies exist between simulated and real measurements, likely due to simplifications in material modeling and actuation dynamics not yet fully supported in the simulation platform, the framework successfully captured the relative performance trends based on the geometry of the design.

VI. CONCLUSION

In this work, we developed a simulation framework that enables the design and dynamic modeling of origami mechanisms directly within a physics-based environment. Our system converts user-defined GUI designs into deformable MuJoCo models, automatically generating meshes, fold constraints, and actuation parameters defined by the user for dynamic simulation. We demonstrated that the framework can accurately capture folding behaviors and interactions with the environment, allowing for both visualization and functional analysis of origami-based robots. To illustrate its utility for design exploration, we applied the framework to optimize an origami catapult mechanism with the objective of maximizing projectile distance. The initial design, mid-optimization design, and resulting optimized design were then tested in hardware models, and the experimental results preserved the same relative performance ranking as observed in simulation.

Although some quantitative discrepancies exist between simulated and real distances, this sim-to-real gap is a common challenge. In our case, we attribute these differences primarily to unmodeled factors such as material properties (e.g., Young's modulus, crease stiffness, and local spring forces), manufacturing imperfections, and the abstraction of actuation in simulation, limitations that stem partly from the current parameterization supported by the MuJoCo deformable model we used. While the flex model captures in-plane deformation and dynamic coupling between connected panels, it does not allow direct specification of crease stiffness or bending energy in the current version of the software. Consequently, folds behave as compliant connections mediated through shared key points rather than as controllable hinges, which limits the software's ability to model crease integrity or programmable folding stiffness. Although there exist more origami-specific simulators that enable detailed modeling of creases, materials, and folding mechanics, our framework prioritizes dynamic interaction, allowing origami mechanisms to be simulated alongside rigid bodies, actuators, and sensors within a unified physics-based environment.

The consistent relative ranking between simulated and hardware designs validates the framework's predictive accuracy for comparative design evaluation. As an extension of this work, we aim to improve the physical fidelity of our models through better characterization of material and fold properties, as well as to explore integration with learning-based or model-driven optimization methods. Ultimately, we envision this framework as a general-purpose platform for rapid, simulation-driven design and analysis of origami-inspired robotic mechanisms, encompassing both compliant and rigid-interactive structures, paving the way for studying origami mechanisms in complex, task-oriented contexts.

REFERENCES

[1] N. Turner, B. Goodwine, and M. Sen, "A review of origami applications in mechanical engineering," *Proceedings of the Institution of Mechanical Engineers, Part C: Journal of Mechanical Engineering Science*, vol. 230, no. 14, pp. 2345–2362, 2016.

[2] D. Rus and M. T. Tolley, "Design, fabrication and control of origami robots," *Nature Reviews Materials*, vol. 3, no. 6, pp. 101–112, 2018.

[3] E. Gultepe, J. S. Randhawa, S. Kadam, *et al.*, "Biopsy with thermally-responsive untethered microtools," *Advanced Materials*, vol. 25, no. 4, pp. 514–519, Jan. 2013, Epub 2012 Oct 9. DOI: 10.1002/adma.201203348.

[4] M. Johnson, Y. Chen, S. Hovet, *et al.*, "Fabricating biomedical origami: A state-of-the-art review," *International journal of computer assisted radiology and surgery*, vol. 12, no. 11, pp. 2023–2032, 2017.

[5] A. J. Taylor, Y. Chen, M. Fok, A. Berman, K. Nilsson, and Z. T. Ho Tse, "Cardiovascular catheter with an expandable origami structure," *Journal of medical devices*, vol. 11, no. 3, p. 034 505, 2017.

[6] A. Lamoureux, K. Lee, M. Shlian, S. R. Forrest, and M. Shtein, "Dynamic kirigami structures for integrated solar tracking," *Nature communications*, vol. 6, no. 1, p. 8092, 2015.

[7] S. Felton, M. Tolley, E. Demaine, D. Rus, and R. Wood, "A method for building self-folding machines," *Science*, vol. 345, no. 6197, pp. 644–646, 2014.

[8] A. Schulz, C. Sung, A. Spielberg, *et al.*, "Interactive robogami: An end-to-end system for design of robots with ground locomotion," *The International Journal of Robotics Research*, vol. 36, no. 10, pp. 1131–1147, 2017. DOI: 10.1177/0278364917723465. eprint: https://doi.org/10.1177/0278364917723465. [Online]. Available: https://doi.org/10.1177/0278364917723465.

[9] Y. Shi, L. Liu, X. Lou, Y. Lu, P. Zhang, and E. Liu, "Oribot: A novel origami robot creation system to support children's steam learning," *Multimedia Tools and Applications*, vol. 84, no. 26, pp. 31 723–31 748, 2025.

[10] C. D. Onal, R. J. Wood, and D. Rus, "An origami-inspired approach to worm robots," *IEEE/ASME transactions on mechatronics*, vol. 18, no. 2, pp. 430–438, 2012.

[11] J.-S. Koh and K.-J. Cho, "Omega-shaped inchworm-inspired crawling robot with large-index-and-pitch (lip) sma spring actuators," *IEEE/ASME Transactions On Mechatronics*, vol. 18, no. 2, pp. 419–429, 2012.

[12] A. T. Baisch, O. Ozcan, B. Goldberg, D. Ithier, and R. J. Wood, "High speed locomotion for a quadrupedal microrobot," *The International Journal of Robotics Research*, vol. 33, no. 8, pp. 1063–1082, 2014.

[13] A. Firouzeh and J. Paik, "Robogami: A fully integrated low-profile robotic origami," *Journal of Mechanisms and Robotics*, vol. 7, no. 2, p. 021 009, 2015.

[14] Z. Zhakypov and J. Paik, "Design methodology for constructing multimaterial origami robots and machines," *IEEE Transactions on Robotics*, vol. 34, no. 1, pp. 151–165, 2018.

[15] B. Chen, Z. Shao, Z. Xie, *et al.*, "Soft origami gripper with variable effective length," *Advanced Intelligent Systems*, vol. 3, no. 10, p. 2 000 251, 2021.

[16] S. Li, J. J. Stampfli, H. J. Xu, *et al.*, "A vacuum-driven origami "magic-ball" soft gripper," in *2019 International Conference on Robotics and Automation (ICRA)*, IEEE, 2019, pp. 7401–7408.

[17] D. Jeong and K. Lee, "Design and analysis of an origami-based three-finger manipulator," *Robotica*, vol. 36, no. 2, pp. 261–274, 2018.

[18] W. Xue, B. Jian, L. Jin, R. Wang, and Q. Ge, "Origami robots: Design, actuation, and 3d printing methods," *Advanced Materials Technologies*, e00278, 2025.

[19] K. Xi, S. Chai, J. Ma, and Y. Chen, "Multi-stability of the extensible origami structures," *Advanced Science*, vol. 10, no. 29, e2303454, Oct. 2023, Epub 2023 Aug 8. DOI: 10.1002/advs.202303454.

[20] H. Chen, Y. Ma, and W. Chen, "Design and optimization of an origami-inspired foldable pneumatic actuator," *IEEE Robotics and Automation Letters*, vol. 9, no. 2, pp. 1278–1285, 2024. DOI: 10.1109/LRA.2023.3343624.

[21] H. Cao, J. Zhou, K. Chen, Q. He, Q. Dou, and Y.-H. Liu, "Design and optimization of an origami gripper for versatile grasping and manipulation," *Advanced Intelligent Systems*, vol. 6, no. 12, p. 2 400 271, 2024.

[22] K. Zhu, H. Guo, W. Yu, *et al.*, "Theoretical modeling and bio-inspired trajectory optimization of a multiple-locomotion origami robot," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 11 355–11 361. DOI: 10.1109/IROS58592.2024.10801958.

[23] Z. Chen, L. Chen, H. Zhang, and J. Zhao, "Co-optimizing physical reconfiguration parameters and controllers for an origami-inspired reconfigurable manipulator," *arXiv preprint arXiv:2504.10474*, 2025.

[24] E. D. Demaine and J. O'Rourke, *Geometric Folding Algorithms: Linkages, Origami, Polyhedra*. New York, NY, USA: Cambridge University Press, 2007, ISBN: 9780521848739.

[25] T. Tachi, "Origamizing polyhedral surfaces," *IEEE transactions on visualization and computer graphics*, vol. 16, no. 2, pp. 298–311, 2009.

[26] T. Tachi, "Simulation of rigid origami," *Origami*, vol. 4, no. 08, pp. 175–187, 2009.

[27] T. Tachi, *Rigid origami simulator*, http://www.tsg.ne.jp/TT/software/, Accessed: October 2025.

[28] T. Tachi, *Freeform origami*, http://www.tsg.ne.jp/TT/software/, Accessed: October 2025.

[29] A. Ghassaei, E. D. Demaine, and N. Gershenfeld, "Fast, interactive origami simulation using gpu computation," *Origami*, vol. 7, pp. 1151–1166, 2018.

[30] C. R. Sung, K. Kuribayashi-Shigetomi, B. An, M. Boyvat, and D. Rus, "A computational design tool for compliant mechanisms based on origami," in *Proceedings of the ASME 2015 International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, Boston, Massachusetts, USA: ASME, Aug. 2015, V05AT08A031. DOI: 10.1115/DETC2015-46982.

[31] Y. Zhu, M. Schenk, and E. Filipov, "A review on origami simulations: From kinematics, to mechanics, towards multiphysics," *Applied Mechanics Reviews*, vol. 74, Jul. 2022. DOI: 10.1115/1.4055031.

[32] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 5026–5033. DOI: 10.1109/IROS.2012.6386109.

[33] Anonymous, *Origami simulation framework code repository*, https://anonymous.4open.science/r/Origami-MuJoCo-Simulation-GUI/README.md, Accessed: 2025-10-30, 2025.

[34] X. Kong, H. Everett, and G. Toussaint, "The graham scan triangulates simple polygons," *Pattern Recognition Letters*, vol. 11, no. 11, pp. 713–716, 1990, ISSN: 0167-8655. DOI: https://doi.org/10.1016/0167-8655(90)90089-K. [Online]. Available: https://www.sciencedirect.com/science/article/pii/016786559090089K.

[35] D.-T. Lee and B. J. Schachter, "Two algorithms for constructing a delaunay triangulation," *International Journal of Computer & Information Sciences*, vol. 9, no. 3, pp. 219–242, 1980.

[36] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 5026–5033. DOI: 10.1109/IROS.2012.6386109.